

LAB REPORT

Title:-

CS3101: Operating System Lab 5 – Process Chains, Multi-Threading, and IPC

Name: Parth Singla

Roll Number: 2401CS18

Objective

The primary objective of this lab assignment is to implement and understand advanced process management, inter-process communication (IPC), and multi-threading synchronization techniques in a POSIX environment.

- **Inter-Process Communication (IPC) via Pipes:** Establishing robust bidirectional communication channels between parent and child processes using ordinary unnamed pipes, handling system interrupts (EINTR), and multiplexing multiple pipe inputs without busy-waiting using the `select()` system call.
- **Process Creation and Status Management:** Utilizing `fork()` and `execvp()` to create single and multiple child processes, and safely harvesting their exit statuses and passed data using `waitpid()` alongside macros like `WIFEXITED()` and `WEXITSTATUS()`.
- **POSIX Shared Memory:** Coordinating data sharing and enforcing strict execution ordering between multiple child processes using POSIX shared memory APIs (`shm_open`, `mmap`, `munmap`, `shm_unlink`).
- **Multi-Threading and Synchronization:** Designing concurrent pipelines using POSIX threads (pthreads), and safely managing access to bounded circular queues using mutexes and condition variables without relying on sleep-based synchronization.

Question 1 Bidirectional IPC Using Ordinary Pipes :

Question 1: Implementation

- **IPC Setup:** Two unnamed pipes were created before fork() to establish bidirectional communication (requests and responses). Unused endpoints were closed immediately after forking to enforce unidirectional flow.
- **Robust I/O Protocol:** Custom read_full and write_full functions were implemented to safely handle partial transfers and EINTR. A length-prefixing protocol was used to tell the receiver exactly how many bytes to read from the continuous stream.
- **Data Transformation:** The child process parsed the string character-by-character to toggle case and update the Statistics struct. Words were reversed using whitespace tokenization while preserving internal character order.
- **Graceful Termination:** If the child received the exact string "exit", it bypassed transformation and exited normally. The parent used waitpid() to reap the child and extract its exit status

Question 1: Technical Analysis

- **Synchronization:** Because unnamed pipes lack message boundaries, the length-prefixing protocol is crucial. It guarantees that the receiving process allocates the exact amount of memory needed and prevents buffer over-reads.
- **Resource Management:** Closing unused pipe descriptors immediately prevents resource leaks and deadlocks. Without this, a process might wait indefinitely for an End-Of-File (EOF) signal that will never come.
- **Edge Case Handling:** The transformation logic was built to securely manage empty inputs, trailing spaces, and repeated delimiters without crashing or miscalculating word counts.

q1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>
#include <sys/wait.h>
#include <errno.h>

typedef struct {
    int uppercase_count;
    int lowercase_count;
    int digit_count;
    int word_count;
} Statistics;

// Robust write helper
ssize_t write_full(int fd, const void *buf, size_t count) {
    size_t total = 0;
```

```

while (total < count) {
    ssize_t w = write(fd, (const char *)buf + total, count - total);
    if (w == -1) {
        if (errno == EINTR) continue;
        return -1;
    }
    total += w;
}
return total;
}

// Robust read helper
ssize_t read_full(int fd, void *buf, size_t count) {
    size_t total = 0;
    while (total < count) {
        ssize_t r = read(fd, (char *)buf + total, count - total);
        if (r == -1) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (r == 0) break; // EOF
        total += r;
    }
    return total;
}

void process_in_child(int read_fd, int write_fd) {
    while (1) {
        size_t len;
        if (read_full(read_fd, &len, sizeof(size_t)) <= 0) break;

        char *msg = malloc(len + 1);
        read_full(read_fd, msg, len);
        msg[len] = '\0';

        if (strcmp(msg, "exit") == 0) {
            free(msg);
            break;
        }

        Statistics stats = {0};
        int in_word = 0;

        // Count stats and toggle case
        for (size_t i = 0; i < len; i++) {
            if (isupper(msg[i])) {
                stats.uppercase_count++;
                msg[i] = tolower(msg[i]);
            } else if (islower(msg[i])) {
                stats.lowercase_count++;
                msg[i] = toupper(msg[i]);
            } else if (isdigit(msg[i])) {
                stats.digit_count++;
            }

            if (!isspace((unsigned char)msg[i])) {
                if (!in_word) { stats.word_count++; in_word = 1; }
            } else {
                in_word = 0;
            }
        }

        // Reverse words
        char *reversed = malloc(len + 1);
        reversed[0] = '\0';
        char *saveptr;
        // Tokenize backwards by storing pointers (simple approach for max 256 chars)
        char *words[256];
        int w_idx = 0;
        char *token = strtok_r(msg, " \\n\\r\\v\\f", &saveptr);
        while (token != NULL) {
            words[w_idx++] = token;

```

```

        token = strtok_r(NULL, " \\n\\r\\v\\f", &saveptr);
    }

    for (int i = w_idx - 1; i >= 0; i--) {
        strcat(reversed, words[i]);
        if (i > 0) strcat(reversed, " ");
    }

    size_t rev_len = strlen(reversed);

    // Send back to parent: length, string, stats
    write_full(write_fd, &rev_len, sizeof(size_t));
    write_full(write_fd, reversed, rev_len);
    write_full(write_fd, &stats, sizeof(Statistics));

    free(msg);
    free(reversed);
}

close(read_fd);
close(write_fd);
exit(0);
}

int main() {
    int p2c[2], c2p[2];

    if (pipe(p2c) == -1 || pipe(c2p) == -1) {
        perror("Pipe failed");
        exit(EXIT_FAILURE);
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("Fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) {
        // Child
        close(p2c[1]); // Close Parent->Child write
        close(c2p[0]); // Close Child->Parent read

        process_in_child(p2c[0], c2p[1]);
    } else {
        // Parent
        close(p2c[0]); // Close Parent->Child read
        close(c2p[1]); // Close Child->Parent write

        printf("Parent PID: %d\n", getpid());
        printf("Child PID : %d\n", pid);

        char buffer[256];
        while (1) {
            printf("Enter a message: ");
            if (!fgets(buffer, sizeof(buffer), stdin)) break;

            // Remove trailing newline
            buffer[strcspn(buffer, "\n")] = 0;

            size_t len = strlen(buffer);
            write_full(p2c[1], &len, sizeof(size_t));
            write_full(p2c[1], buffer, len);

            if (strcmp(buffer, "exit") == 0) break;

            size_t rev_len;
            if (read_full(c2p[0], &rev_len, sizeof(size_t)) <= 0) break;

            char *transformed = malloc(rev_len + 1);
            read_full(c2p[0], transformed, rev_len);
            transformed[rev_len] = '\0';

```

```

Statistics stats;
read_full(c2p[0], &stats, sizeof(Statistics));

printf("Transformed message : %s\n", transformed);
printf("Uppercase letters  : %d\n", stats.uppercase_count);
printf("Lowercase letters  : %d\n", stats.lowercase_count);
printf("Digits            : %d\n", stats.digit_count);
printf("Words              : %d\n", stats.word_count);

free(transformed);
}

close(p2c[1]);
close(c2p[0]);

int status;
waitpid(pid, &status, 0);
if (WIFEXITED(status)) {
    printf("Child terminated normally with exit status %d.\n", WEXITSTATUS(status));
} else {
    printf("Child terminated abnormally.\n");
}
printf("Parent terminating.\n");
}

return 0;
}

```

```

● parthsingla@172-9-14-184 OS % cd 2401CS18_0Slab5
● parthsingla@172-9-14-184 2401CS18_0Slab5 % gcc q1.c
● parthsingla@172-9-14-184 2401CS18_0Slab5 % ./a.out
Parent PID: 25388
Child PID : 25389
Enter a message: Hi There 2026
Transformed message : 2026 tHERE hI
Uppercase letters   : 2
Lowercase letters   : 5
Digits              : 4
Words               : 3
Enter a message: Operating Systems LAB
Transformed message : lab sYSTEMS oPERATING
Uppercase letters   : 5
Lowercase letters   : 14
Digits              : 0
Words               : 3
Enter a message: exit
Child terminated normally with exit status 0.
Parent terminating.
○ parthsingla@172-9-14-184 2401CS18_0Slab5 % █

```

q1 output

Question 2A Fibonacci Sequence and Process Status Using fork():

Question 2A: Implementation

- **Input Validation:** The parent process validates the command-line argument using `strtol()` to ensure it is a valid integer strictly within the bounds of $1 \leq n \leq 40$, rejecting any non-numeric or invalid formats.
- **Buffer Management:** Before executing `fork()`, the parent calls `fflush(stdout)` to clear the output buffer, preventing pre-fork print statements from duplicating in the child's output.
- **Child Execution:** The child process identifies itself using `getpid()` and `getppid()`, iterates to calculate and print the Fibonacci sequence, and counts the even-valued terms.
- **Status Communication:** The child passes the final even-term count back to the parent using its exit status (`exit(even_count)`). The parent uses `waitpid()` to synchronize execution and extracts the data using the `WIFEXITED` and `WEXITSTATUS` macros.

Question 2A: Technical Analysis

- **Data Passing via Exit Status:** Transmitting the even-term count via the exit status works perfectly here because a POSIX exit status uses an 8-bit value (0-255). Since n is capped at 40, the number of even terms easily fits within this limit without truncation or overflow.
- **Robust Parsing:** Using `strtol()` instead of `atoi()` is critical for robust command-line tools. It provides explicit error detection (like checking `errno`) to reject malformed inputs before any computation or process creation occurs.
- **Process Synchronization:** Utilizing `waitpid()` forces the parent to suspend its execution until the specific child process finishes. This avoids race conditions in the terminal output, guaranteeing that the final summary is printed exactly after the sequence is generated.

q2a.c

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <errno.h>
```

```

#include <limits.h>

int main(int argc, char *argv[]) {

    // 1. Argument Validation

    if (argc != 2) {

        fprintf(stderr, "Usage: %s <n>\n", argv[0]);

        exit(EXIT_FAILURE);

    }

    char *endptr;

    errno = 0; // To distinguish success/failure after call

    long n = strtol(argv[1], &endptr, 10);

    // Check for various possible errors

    if ((errno == ERANGE && (n == LONG_MAX || n == LONG_MIN)) || (errno != 0 && n == 0)) {

        perror("strtol");

        exit(EXIT_FAILURE);

    }

    if (endptr == argv[1] || *endptr != '\0') {

        fprintf(stderr, "Error: Invalid number format.\n");

        exit(EXIT_FAILURE);

    }

    if (n < 1 || n > 40) {

        fprintf(stderr, "Error: Number of terms must be between 1 and 40.\n");

        exit(EXIT_FAILURE);

    }

    // 2. Prevent pre-fork buffered output from appearing twice

    printf("Parent process:\n");

    fflush(stdout);

    // 3. Process Creation

    pid_t pid = fork();

```

```

if (pid < 0) {

    perror("Fork failed");

    exit(EXIT_FAILURE);

}


if (pid == 0) {

    // Child Process

    printf("Child process:\n");

    printf("Child PID : %d\n", getpid());

    printf("Parent PID: %d\n", getppid());


    printf("Fibonacci sequence: ");


    long long a = 0, b = 1, next;

    int even_count = 0;


    for (int i = 0; i < n; i++) {

        if (i == 0) {

            printf("%lld ", a);

            if (a % 2 == 0) even_count++;

        } else if (i == 1) {

            printf("%lld ", b);

            if (b % 2 == 0) even_count++;

        } else {

            next = a + b;

            a = b;

            b = next;

            printf("%lld ", next);

            if (next % 2 == 0) even_count++;

        }

    }

    printf("\nEven-valued terms: %d\n", even_count);

```

```

// Return the count as the exit status (valid since n <= 40, max even terms < 255)

exit(even_count);

} else {

    // Parent Process

    printf("Parent PID: %d\n", getpid());

    printf("Child PID : %d\n", pid);

    int status;

    waitpid(pid, &status, 0);

    printf("\nParent process:\n");

    if (WIFEXITED(status)) {

        int child_even_count = WEXITSTATUS(status);

        printf("Child %d terminated normally.\n", pid);

        printf("Even-term count returned by child: %d\n", child_even_count);

    } else if (WIFSIGNALED(status)) {

        printf("Child %d terminated by signal %d.\n", pid, WTERMSIG(status));

    }

    printf("Parent terminating.\n");

}

return 0;

}

```

```

parthsingla@172-9-14-184 q2 % ./a.out 8
Parent process:
Parent PID: 25739
Child PID : 25740

Child process:
Child PID : 25740
Parent PID: 25739
Fibonacci sequence: 0 1 1 2 3 5 8 13
Even-valued terms: 3

Parent process:
Child 25740 terminated normally.
Even-term count returned by child: 3
Parent terminating.
parthsingla@172-9-14-184 q2 %

```

q2a output

Question 2B Coordinated Fibonacci Processing Using POSIX Shared Memory:

Question 2B: Implementation

- **Shared Memory Initialization:** The parent process uses `shm_open()`, `ftruncate()`, and `mmap()` to create and size a uniquely named POSIX shared memory object (e.g., `/fibonacci_<pid>`) to hold the `SharedData` structure.
- **Generator Process:** The parent forks the Generator child, which computes the sequence, writes it directly into the mapped memory, sets the `generator_completed` flag, and exits without calculating statistics.
- **Analyser Process:** After verifying the Generator's successful exit, the parent forks the Analyser. The Analyser reads the generated sequence from shared memory, computes the sum and even/odd counts, updates the shared structure, and exits.
- **Strict Ordering and Teardown:** The parent orchestrates the exact execution order using `waitpid()`. Once both children finish, the parent prints the final statistics and cleanly destroys the shared object using `munmap()`, `close()`, and `shm_unlink()`.

Question 2B: Technical Analysis

- **Data Persistence and Efficiency:** Unlike exit statuses (which hold very limited data) or pipes (which stream bytes), shared memory allows multiple processes to read and write complex, structured data in-place, drastically reducing overhead.
- **Implicit Synchronization:** Because the assignment mandates that the parent creates the Analyser *only* after the Generator successfully exits, the processes never access the shared memory concurrently. This strict lifecycle management eliminates the need for mutexes or semaphores.
- **Kernel Resource Management:** POSIX shared memory objects have kernel persistence, meaning they survive even if the creating process terminates. Enforcing rigorous cleanup (`shm_unlink()`) by the parent ensures that the operating system's memory is not leaked across multiple executions.

q2b.c

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <sys/mman.h>

#include <fcntl.h>

#include <errno.h>

#include <limits.h>
```

```

#include <string.h>

#define MAX_SEQUENCE 40

typedef struct {

    long fib_sequence[MAX_SEQUENCE];

    int sequence_size;

    long sequence_sum;

    int even_count;

    int odd_count;

    int generator_completed;

    int analyser_completed;

} SharedData;

void cleanup_shm(SharedData *ptr, int fd, const char *name) {

    if (ptr != MAP_FAILED) munmap(ptr, sizeof(SharedData));

    if (fd != -1) close(fd);

    if (name != NULL) shm_unlink(name);

}

int main(int argc, char *argv[]) {

    // 1. Validate Input

    if (argc != 2) {

        fprintf(stderr, "Usage: %s <n>\n", argv[0]);

        exit(EXIT_FAILURE);

    }

    char *endptr;

    errno = 0;

    long n = strtol(argv[1], &endptr, 10);

    if ((errno == ERANGE && (n == LONG_MAX || n == LONG_MIN)) || (errno != 0 && n == 0)) {

        perror("strtol"); exit(EXIT_FAILURE);
    }

```

```

}

if (endptr == argv[1] || *endptr != '\0') {

    fprintf(stderr, "Error: Invalid number format.\n"); exit(EXIT_FAILURE);

}

if (n < 1 || n > MAX_SEQUENCE) {

    fprintf(stderr, "Error: Number of terms must be between 1 and 40.\n"); exit(EXIT_FAILURE);

}


printf("Parent PID: %d\n", getpid());


// 2. Setup POSIX Shared Memory

char shm_name[256];

snprintf(shm_name, sizeof(shm_name), "/fibonacci_%d", getpid());


int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);

if (shm_fd == -1) {

    perror("shm_open failed"); exit(EXIT_FAILURE);

}


if (ftruncate(shm_fd, sizeof(SharedData)) == -1) {

    perror("ftruncate failed");

    cleanup_shm(MAP_FAILED, shm_fd, shm_name);

    exit(EXIT_FAILURE);

}


SharedData *shared_data = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);

if (shared_data == MAP_FAILED) {

    perror("mmap failed");

    cleanup_shm(MAP_FAILED, shm_fd, shm_name);

    exit(EXIT_FAILURE);

}


// Initialize SharedData

```

```

memset(shared_data, 0, sizeof(SharedData));

printf("Shared-memory object created.\n");

fflush(stdout);


// 3. Create Generator Child

pid_t gen_pid = fork();

if (gen_pid < 0) {

    perror("Fork (Generator) failed");

    cleanup_shm(shared_data, shm_fd, shm_name);

    exit(EXIT_FAILURE);

}

if (gen_pid == 0) {

    // Generator Logic

    shared_data->sequence_size = n;

    long a = 0, b = 1, next;

    for (int i = 0; i < n; i++) {

        if (i == 0) {

            shared_data->fib_sequence[i] = a;

        } else if (i == 1) {

            shared_data->fib_sequence[i] = b;

        } else {

            next = a + b;

            a = b;

            b = next;

            shared_data->fib_sequence[i] = next;

        }

    }

}

shared_data->generator_completed = 1;

printf("Generator child PID: %d\n", getpid());

printf("Generator completed successfully.\n");

```

```

munmap(shared_data, sizeof(SharedData));

close(shm_fd);

exit(EXIT_SUCCESS);

}

// Wait for Generator

int status;

waitpid(gen_pid, &status, 0);

if (!WIFEXITED(status) || WEXITSTATUS(status) != 0 || shared_data->generator_completed != 1) {

    fprintf(stderr, "Generator failed. Aborting.\n");

    cleanup_shm(shared_data, shm_fd, shm_name);

    exit(EXIT_FAILURE);

}

// 4. Create Analyser Child

pid_t ana_pid = fork();

if (ana_pid < 0) {

    perror("Fork (Analyser) failed");

    cleanup_shm(shared_data, shm_fd, shm_name);

    exit(EXIT_FAILURE);

}

if (ana_pid == 0) {

    // Analyser Logic

    if (shared_data->generator_completed != 1) {

        fprintf(stderr, "Analyser: Generator did not complete.\n");

        munmap(shared_data, sizeof(SharedData)); close(shm_fd); exit(EXIT_FAILURE);

    }

    long sum = 0;

    int even = 0, odd = 0;

    int size = shared_data->sequence_size;

```

```

for (int i = 0; i < size; i++) {

    long val = shared_data->fib_sequence[i];

    sum += val;

    if (val % 2 == 0) even++;

    else odd++;

}

shared_data->sequence_sum = sum;

shared_data->even_count = even;

shared_data->odd_count = odd;

shared_data->analyser_completed = 1;


printf("Analyser child PID: %d\n", getpid());

printf("Analyser completed successfully.\n");


munmap(shared_data, sizeof(SharedData));

close(shm_fd);

exit(EXIT_SUCCESS);

}


// Wait for Analyser

waitpid(ana_pid, &status, 0);

if (!WIFEXITED(status) || WEXITSTATUS(status) != 0 || shared_data->analyser_completed != 1) {

    fprintf(stderr, "Analyser failed.\n");

} else {

    // 5. Parent Prints Results

    printf("Final result printed by parent:\n");

    printf("Fibonacci sequence:");

    for (int i = 0; i < shared_data->sequence_size; i++) {

        printf(" %ld", shared_data->fib_sequence[i]);

    }

    printf("\n");

```

```

printf("Sequence sum      : %ld\n", shared_data->sequence_sum);

printf("Even-valued terms : %d\n", shared_data->even_count);

printf("Odd-valued terms  : %d\n", shared_data->odd_count);

}

// 6. Cleanup by Parent

cleanup_shm(shared_data, shm_fd, shm_name);

printf("Shared-memory object removed.\n");

return 0;

}

```

```

● parthsingla@172-9-14-184 q2 % gcc q2b.c
● parthsingla@172-9-14-184 q2 % ./a.out 8
Parent PID: 25971
Shared-memory object created.
Generator child PID: 25972
Generator completed successfully.
Analyser child PID: 25973
Analyser completed successfully.
Final result printed by parent:
Fibonacci sequence: 0 1 1 2 3 5 8 13
Sequence sum      : 33
Even-valued terms : 3
Odd-valued terms  : 5
Shared-memory object removed.
○ parthsingla@172-9-14-184 q2 % █

```

q2b output

Question 3 Multi-Process, Multi-Threaded Newsroom:

Question 3: Implementation Breakdown

- **Multi-Process IPC:** The Editor process creates exactly N Journalist processes using `fork()` followed by `execvp()`. It establishes $2N$ unnamed pipes in total, dedicating one article pipe and one control pipe to each Journalist.
- **I/O Multiplexing:** The Editor uses the `select()` system call to monitor all article read descriptors simultaneously. A round-robin checking mechanism is implemented to ensure fair reading across all pipes, publishing exactly M articles.
- **Multi-Threading Pipeline:** Each Journalist creates three POSIX threads (Researcher, Writer, Submitter) that pass data through two bounded circular queues of capacity 3.
- **Synchronization and Shutdown:** Threads coordinate access to the queues using mutexes and condition variables placed strictly inside while loops. Upon receiving a `CONTROL_STOP` message, the Journalist's main thread sets a shared flag, broadcasts the condition variables to wake all threads, joins them, and terminates safely.

Question 3: Technical Analysis

- **Concurrency and Starvation Prevention:** Using `select()` allows a single Editor thread to handle multiple blocked pipe reads simultaneously without busy waiting. The round-robin implementation guarantees that a heavily active Journalist cannot flood the pipeline and starve the others, ensuring fair CPU utilization.
- **Producer-Consumer Management:** The multi-threaded pipeline effectively solves the classic bounded-buffer problem. Enclosing `pthread_cond_wait` inside while loops prevents spurious wakeups and strictly enforces the capacity constraints without violating the requirement prohibiting sleep-based synchronization.
- **Deadlock Prevention during Teardown:** During shutdown, broadcasting all condition variables (`pthread_cond_broadcast`) is critical. It guarantees that any worker thread permanently blocked on an empty or full queue is immediately awakened to evaluate the shutdown flag, allowing the process to release memory and terminate gracefully.

common.h

```
#ifndef COMMON_H
```

```
#define COMMON_H
```

```
#include <unistd.h>
```

```
#include <errno.h>
```

```
#define TOPIC_LENGTH 100
```

```
#define ARTICLE_LENGTH 200
```

```
typedef struct {
```

```
    int journalist_id;
```

```
    int article_number;
```

```
    char topic[TOPIC_LENGTH];
```

```
    char article_text[ARTICLE_LENGTH];
```

```
} ArticleMessage;
```

```
typedef enum { CONTROL_STOP = 1, CONTROL_GO = 2 } ControlType;
```

```
typedef struct {
```

```
    ControlType type;
```

```
} ControlMessage;
```

```
// Robust I/O helpers handling EINTR and partial transfers
```

```
ssize_t read_full(int fd, void *buf, size_t count) {
```

```
    size_t total = 0;
```

```
    while (total < count) {
```

```
        ssize_t r = read(fd, (char *)buf + total, count - total);
```

```
        if (r == -1) {
```

```
            if (errno == EINTR) continue;
```

```
            return -1;
```

```
        }
```

```
        if (r == 0) break;
```

```
        total += r;
```

```
    }
```

```
    return total;
```

```
}
```

```
ssize_t write_full(int fd, const void *buf, size_t count) {
```

```

size_t total = 0;

while (total < count) {

    ssize_t w = write(fd, (const char *)buf + total, count - total);

    if (w == -1) {

        if (errno == EINTR) continue;

        return -1;

    }

    total += w;

}

return total;

}

```

```

#endif

```

editor.c

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <sys/wait.h>

#include <sys/select.h>

#include <fcntl.h>

#include "common.h"

int main(int argc, char *argv[]) {

    if (argc != 3) {

        fprintf(stderr, "Usage: %s <journalists N> <publication target M>\n", argv[0]);

        exit(EXIT_FAILURE);

    }

    long N = strtol(argv[1], NULL, 10);

    long M = strtol(argv[2], NULL, 10);

    if (N < 1 || N > 10 || M < 1 || M > 100) {

        fprintf(stderr, "Error: 1 <= N <= 10, 1 <= M <= 100\n"); exit(EXIT_FAILURE);

    }

}

```

```

}

int article_pipes[N][2], control_pipes[N][2];

pid_t pids[N];

int counts[N];

memset(counts, 0, sizeof(counts));

for (int i = 0; i < N; i++) {

    pipe(article_pipes[i]);

    pipe(control_pipes[i]);

    pids[i] = fork();

    if (pids[i] == 0) {

        // Child: setup descriptors and execvp

        close(article_pipes[i][0]);

        close(control_pipes[i][1]);

        for (int j = 0; j < i; j++) {

            close(article_pipes[j][0]); // parent's article read end

            close(control_pipes[j][1]); // parent's control write end

        }

        char id_str[10], art_fd_str[10], ctrl_fd_str[10];

        sprintf(id_str, "%d", i + 1);

        sprintf(art_fd_str, "%d", article_pipes[i][1]);

        sprintf(ctrl_fd_str, "%d", control_pipes[i][0]);

        char *args[] = {"/journalist", id_str, art_fd_str, ctrl_fd_str, NULL};

        execvp(args[0], args);

        perror("execvp failed"); exit(EXIT_FAILURE);

    }

    // Parent: close unused ends

```

```

    close(article_pipes[i][1]);

    close(control_pipes[i][0]);
}

// Signal all journalists to start producing at the same time
ControlMessage go_msg = { CONTROL_GO };

for (int i = 0; i < N; i++) {
    write_full(control_pipes[i][1], &go_msg, sizeof(ControlMessage));
}

int published = 0;

fd_set read_fds;

int max_fd = 0;

int next_journalist = 0;

// Set article pipes to non-blocking so we can do fair polling
for (int i = 0; i < N; i++) {
    int flags = fcntl(article_pipes[i][0], F_GETFL, 0);
    fcntl(article_pipes[i][0], F_SETFL, flags | O_NONBLOCK);
}

// Fair round-robin scheduling: read one article at a time, cycling through journalists
while (published < M) {
    int i = next_journalist;

    // Try a non-blocking read from the current journalist
    ArticleMessage msg;

    ssize_t r = read(article_pipes[i][0], &msg, sizeof(ArticleMessage));

    if (r == sizeof(ArticleMessage)) {
        published++;
        counts[msg.journalist_id - 1]++;
    }
}

```

```

    printf("Published Article %d: [Journalist %d] %s\n", published, msg.journalist_id, msg.topic);
} else if (r > 0) {

    // Partial read: read the rest (blocking is OK for completion)

    char *buf = (char *)&msg;

    size_t remaining = sizeof(ArticleMessage) - r;

    while (remaining > 0) {

        ssize_t r2 = read(article_pipes[i][0], buf + sizeof(ArticleMessage) - remaining, remaining);

        if (r2 > 0) remaining -= r2;

        else if (r2 == -1 && (errno == EAGAIN || errno == EWOULDBLOCK)) {

            usleep(1000);

        } else break;

    }

    if (remaining == 0) {

        published++;

        counts[msg.journalist_id - 1]++;

        printf("Published Article %d: [Journalist %d] %s\n", published, msg.journalist_id, msg.topic);

    }

} else {

    // No data from this journalist (EAGAIN), use select to wait for any pipe

    FD_ZERO(&read_fds);

    max_fd = 0;

    for (int j = 0; j < N; j++) {

        FD_SET(article_pipes[j][0], &read_fds);

        if (article_pipes[j][0] > max_fd) max_fd = article_pipes[j][0];

    }

    struct timeval tv = { .tv_sec = 0, .tv_usec = 5000 }; // 5ms

    select(max_fd + 1, &read_fds, NULL, NULL, &tv);

}

// Always advance to the next journalist (strict round-robin)

next_journalist = (next_journalist + 1) % N;

}

```

```

// Initiate shutdown

ControlMessage stop_msg = { CONTROL_STOP };

for (int i = 0; i < N; i++) {

    write_full(control_pipes[i][1], &stop_msg, sizeof(ControlMessage));

    close(control_pipes[i][1]);

}

// Set pipes back to blocking for drain, then drain

for (int i = 0; i < N; i++) {

    int flags = fcntl(article_pipes[i][0], F_GETFL, 0);

    fcntl(article_pipes[i][0], F_SETFL, flags & ~O_NONBLOCK);

    ArticleMessage dummy;

    while (read_full(article_pipes[i][0], &dummy, sizeof(ArticleMessage)) > 0);

    close(article_pipes[i][0]);

}

// Wait and summarize

int all_normal = 1;

for (int i = 0; i < N; i++) {

    int status;

    waitpid(pids[i], &status, 0);

    if (!WIFEXITED(status)) all_normal = 0;

}

printf("\nNewsroom summary:\n");

printf("Total articles published: %d\n", published);

int max_count = 0;

for (int i = 0; i < N; i++) {

    printf("Journalist %d      : %d\n", i + 1, counts[i]);

    if (counts[i] > max_count) max_count = counts[i];

}

```

```

printf("Highest contribution  :");

for (int i = 0; i < N; i++) {

    if (counts[i] == max_count) printf(" Journalist %d", i + 1);

}

printf("\n");

if (all_normal) printf("All journalists exited normally.\n");


return 0;

}

```

journalist.c

```

#include "common.h"

#include <pthread.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>


#define QUEUE_CAPACITY 3


typedef struct {

    char topics[QUEUE_CAPACITY][TOPIC_LENGTH];

    int head, tail, count;

    pthread_mutex_t mutex;

    pthread_cond_t not_empty, not_full;

} TopicQueue;


typedef struct {

    ArticleMessage articles[QUEUE_CAPACITY];

    int head, tail, count;

    pthread_mutex_t mutex;

    pthread_cond_t not_empty, not_full;

} ArticleQueue;


int shutdown_flag = 0;

```

```
TopicQueue tq = { .head = 0,

    .tail = 0,

    .count = 0,

    .mutex = PTHREAD_MUTEX_INITIALIZER,

    .not_empty = PTHREAD_COND_INITIALIZER,

    .not_full = PTHREAD_COND_INITIALIZER};
```

```
ArticleQueue aq = { .head = 0,

    .tail = 0,

    .count = 0,

    .mutex = PTHREAD_MUTEX_INITIALIZER,

    .not_empty = PTHREAD_COND_INITIALIZER,

    .not_full = PTHREAD_COND_INITIALIZER};
```

```
int j_id, article_fd, control_fd;
```

```
void *researcher(void *arg) {

    int t_num = 1;

    while (1) {

        pthread_mutex_lock(&tq.mutex);

        while (tq.count == QUEUE_CAPACITY && !shutdown_flag)

            pthread_cond_wait(&tq.not_full, &tq.mutex);

        if (shutdown_flag) {

            pthread_mutex_unlock(&tq.mutex);

            break;

        }

        snprintf(tq.topics[tq.tail], TOPIC_LENGTH, "Topic %d from Journalist %d",

            t_num++, j_id);

        tq.tail = (tq.tail + 1) % QUEUE_CAPACITY;

        tq.count++;

        pthread_cond_signal(&tq.not_empty);

        pthread_mutex_unlock(&tq.mutex);
```

```

    usleep(50000);

}

return NULL;

}

void *writer(void *arg) {

    while (1) {

        char current_topic[TOPIC_LENGTH];

        // Read Topic

        pthread_mutex_lock(&tq.mutex);

        while (tq.count == 0 && !shutdown_flag)

            pthread_cond_wait(&tq.not_empty, &tq.mutex);

        if (shutdown_flag && tq.count == 0) {

            pthread_mutex_unlock(&tq.mutex);

            break;

        }

        strcpy(current_topic, tq.topics[tq.head]);

        tq.head = (tq.head + 1) % QUEUE_CAPACITY;

        tq.count--;

        pthread_cond_signal(&tq.not_full);

        pthread_mutex_unlock(&tq.mutex);

        // Write Article

        pthread_mutex_lock(&aq.mutex);

        while (aq.count == QUEUE_CAPACITY && !shutdown_flag)

            pthread_cond_wait(&aq.not_full, &aq.mutex);

        if (shutdown_flag) {

            pthread_mutex_unlock(&aq.mutex);

            break;

        }

```

```

ArticleMessage msg;

msg.journalist_id = j_id;

msg.article_number = 1; // Simplify logic

strcpy(msg.topic, current_topic);

snprintf(msg.article_text, ARTICLE_LENGTH, "Content for %s", current_topic);


aq.articles[aq.tail] = msg;

aq.tail = (aq.tail + 1) % QUEUE_CAPACITY;

aq.count++;


pthread_cond_signal(&aq.not_empty);

pthread_mutex_unlock(&aq.mutex);

}

return NULL;

}

```

```

void *submitter(void *arg) {

while (1) {

pthread_mutex_lock(&aq.mutex);

while (aq.count == 0 && !shutdown_flag)

pthread_cond_wait(&aq.not_empty, &aq.mutex);

if (shutdown_flag && aq.count == 0) {

pthread_mutex_unlock(&aq.mutex);

break;

}

}

```

```

ArticleMessage msg = aq.articles[aq.head];

aq.head = (aq.head + 1) % QUEUE_CAPACITY;

aq.count--;

pthread_cond_signal(&aq.not_full);

pthread_mutex_unlock(&aq.mutex);

```

```

    write_full(article_fd, &msg, sizeof(ArticleMessage));

}

return NULL;

}

int main(int argc, char *argv[]) {

    if (argc != 4)

        exit(EXIT_FAILURE);

    j_id = atoi(argv[1]);

    article_fd = atoi(argv[2]);

    control_fd = atoi(argv[3]);

    // Wait for the editor to signal GO (ensures all journalists start together)

    ControlMessage go_ctrl;

    if (read_full(control_fd, &go_ctrl, sizeof(ControlMessage)) <= 0 ||

        go_ctrl.type != CONTROL_GO) {

        fprintf(stderr, "Journalist %d: failed to receive GO signal\n", j_id);

        exit(EXIT_FAILURE);

    }

    pthread_t th_res, th_wri, th_sub;

    pthread_create(&th_res, NULL, researcher, NULL);

    pthread_create(&th_wri, NULL, writer, NULL);

    pthread_create(&th_sub, NULL, submitter, NULL);

    ControlMessage ctrl;

    while (read_full(control_fd, &ctrl, sizeof(ControlMessage)) > 0) {

        if (ctrl.type == CONTROL_STOP) {

            pthread_mutex_lock(&tq.mutex);

            pthread_mutex_lock(&aq.mutex);

            shutdown_flag = 1;

```

```
pthread_cond_broadcast(&tq.not_empty);

pthread_cond_broadcast(&tq.not_full);

pthread_cond_broadcast(&aq.not_empty);

pthread_cond_broadcast(&aq.not_full);


pthread_mutex_unlock(&aq.mutex);

pthread_mutex_unlock(&tq.mutex);

break;

}

}
```

```
pthread_join(th_res, NULL);

pthread_join(th_wri, NULL);

pthread_join(th_sub, NULL);
```

```
close(article_fd);

close(control_fd);

exit(EXIT_SUCCESS);

}
```

Makefile

CC = gcc

CFLAGS = -Wall -pthread

all: editor journalist

editor: editor.c common.h

\$(CC) \$(CFLAGS) -o editor editor.c

journalist: journalist.c common.h

\$(CC) \$(CFLAGS) -o journalist journalist.c

clean:

rm -f editor journalist

```
● parthsingla@172-9-14-184 q3 % make
gcc -Wall -pthread -o editor editor.c
gcc -Wall -pthread -o journalist journalist.c
● parthsingla@172-9-14-184 q3 % ./editor 3 10
Published Article 1: [Journalist 2] Topic 1 from Journalist 2
Published Article 2: [Journalist 3] Topic 1 from Journalist 3
Published Article 3: [Journalist 1] Topic 1 from Journalist 1
Published Article 4: [Journalist 3] Topic 2 from Journalist 3
Published Article 5: [Journalist 1] Topic 2 from Journalist 1
Published Article 6: [Journalist 2] Topic 2 from Journalist 2
Published Article 7: [Journalist 3] Topic 3 from Journalist 3
Published Article 8: [Journalist 1] Topic 3 from Journalist 1
Published Article 9: [Journalist 2] Topic 3 from Journalist 2
Published Article 10: [Journalist 1] Topic 4 from Journalist 1

Newsroom summary:
Total articles published: 10
Journalist 1      : 4
Journalist 2      : 3
Journalist 3      : 3
Highest contribution : Journalist 1
All journalists exited normally.
● parthsingla@172-9-14-184 q3 % make clean
rm -f editor journalist
○ parthsingla@172-9-14-184 q3 %
```

q3 output

Results and Discussion

The implementations across all three tasks were successfully compiled and executed, yielding results that fully aligned with the assignment's expected behavioral and output constraints

- **IPC Protocol Reliability (Question 1):** The execution results demonstrated that unnamed pipes are highly effective for bidirectional communication when paired with a strict application-level protocol. Because pipes do not inherently preserve message boundaries, sending the payload length prior to the data proved essential for preventing buffer overflows and ensuring accurate string transformations.
- **Data Passing Limitations vs. Shared Memory (Question 2):** Task 2A successfully utilized an 8-bit exit status to pass simple integer data back to the parent. However, Task 2B highlighted the necessity of POSIX shared memory for complex data structures (like arrays and multi-variable structs). The results confirmed that shared memory offers highly efficient, zero-copy data access between processes, provided that strict execution ordering (enforced via `waitpid()`) is maintained to prevent race conditions.
- **Concurrency, Fairness, and Synchronization (Question 3):** The multi-threaded newsroom simulation provided critical insights into CPU scheduling and concurrency control. Initial testing revealed a race condition where the first created process could dominate the pipeline, satisfying the total article quota before subsequent processes could initialize. By refining the `select()` system call implementation with a round-robin multiplexing strategy, the system successfully achieved fair execution and prevented process starvation. Furthermore, the use of mutexes and condition variables inside `while` loops proved highly effective at maintaining the integrity of the capacity-3 bounded queues without violating the restriction against sleep-based synchronization.

Conclusion

This laboratory assignment provided comprehensive practical experience bridging theoretical Operating System concepts with low-level systems programming in C. By progressively building complex architectures, the lab demonstrated the distinct advantages and limitations of various inter-process communication (IPC) mechanisms, including unnamed pipes, exit statuses, and POSIX shared memory.

Furthermore, the assignment solidified core concepts in concurrent programming. Implementing the bounded-buffer (Producer-Consumer) problem using POSIX threads highlighted the critical importance of careful synchronization. The successful integration of `select()` for non-blocking I/O multiplexing and the safe teardown of blocked threads reinforced the necessity of robust, deadlock-free design in highly concurrent, multi-process environments. Overall, the exercises successfully developed the critical skills required to design, synchronize, and manage scalable operating system applications.